Lokalne LLM na Unitree G1

Streszczenie kierownicze

Unitree G1 ma architekturę obliczeniową rozdzieloną na co najmniej dwa komputery: niedostępny dla użytkownika „locomotion computer” (kontrola niskopoziomowa, stabilizacja, sterowniki) oraz dostępny „development computer”, który jest główną bramą do uruchamiania własnego kodu (zarówno wysoki poziom, jak i wybrane sterowanie niskopoziomowe). Ten układ jest krytyczny dla LLM: model nie powinien nigdy wchodzić w ścieżkę sterowania o twardych wymaganiach czasu rzeczywistego, tylko działać asynchronicznie jako warstwa „intencji/planowania” nad zestawem bezpiecznych prymitywów ruchu. [1]

W dokumentacji integracyjnej G1 (nieoficjalnej, ale technicznie spójnej z SDK) wskazano, że komputer developerski w G1 bazuje na NVIDIA Jetson Orin NX na dedykowanej płycie nośnej Unitree (ważne: wymaga specyficznego BSP i nie powinno się go flashować „obrazami zewnętrznymi”). To mocno sugeruje środowisko Linux/ARM64 + CUDA (JetPack), a więc naturalny wybór stanowią: (a) lekkie modele 2–4B w 4-bit, (b) modele klasy 7–9B w 4-bit, ale tylko przy ograniczeniu kontekstu i agresywnej kontroli pamięci, (c) ewentualne hybrydy (część lokalnie, część na serwerze przy krawędzi) przez Ethernet/Wi-Fi. [2]

Z punktu widzenia „LLM na robocie” najbardziej praktyczna konfiguracja to **tekstowy LLM 2–4B w int4** jako model domyślny (niski narzut pamięciowy, wysoka responsywność), z możliwością przełączania na większy model 9B w int4 dla trudniejszych zapytań i planowania wieloetapowego. Wśród nowych rodzin modeli **Qwen3.5 (0.8B/2B/4B/9B, Apache-2.0)** wyróżnia się bardzo dobrą relacją zdolności↔rozmiar oraz oficjalnym wsparciem dla serwowania przez popularne silniki (Transformers, vLLM, SGLang, KTransformers). Jednocześnie Qwen3.5 ma domyślnie ekstremalnie długi kontekst (262k), co na urządzeniu wbudowanym należy eksplicytnie ograniczyć (np. 8k–16k) albo grozi OOM. [3]

Dla ścieżki „speech-to-speech” (pełny dupleks: jednoczesne słuchanie i mówienie) najciekawszą propozycją jest **NVIDIA PersonaPlex (7B)**, ale ma ona cięższe wymagania sprzętowe i licencyjne (NVIDIA Open Model License dla wag) oraz jest projektowana/benchmarkowana pod duże GPU (A100/H100). Może być użyteczna jako opcja „premium”, lecz dla G1 typowo bezpieczniej jest zastosować klasyczny pipeline **ASR → tekstowy LLM → TTS** i dopiero później rozważać PersonaPlex. [4]

Rekomendacja końcowa (praktyczna, wdrażalna na G1 bez „hackowania” sterowania): MVP: Qwen3.5-2B (int4) + fallback Qwen3.5-4B (int4) + „tryb ciężki” Qwen3.5-9B (int4) uruchamiane lokalnie na komputerze developerskim Jetson Orin NX, z twardą bramką bezpieczeństwa (skill-gating, reguły, limity prędkości/sił, timeouts) oraz z możliwością offloadu na serwer edge przy spadku wydajności albo przy bardziej złożonych zadaniach.

Kandydackie modele i rozwiązania lokalne: porównanie

Poniższa tabela celowo miesza „modele tekstowe" oraz „modele konwersacyjne mówione" (PersonaPlex), bo wymagania użytkownika obejmują integrację z mową i percepcją. Tam, gdzie brakuje twardych danych producenta o pamięci/latencji na Jetson Orin NX, podaję **estymacje inżynierskie** oparte o: (a) znane rozmiary wag (parametry), (b) publiczny benchmark llama.cpp na Orin NX 16GB (Llama2 7B Q4_0), (c) typowe reguły skalowania (większy model ≈ niższe tok/s) oraz (d) konieczność ograniczenia kontekstu (KV-cache). Benchmark odniesienia dla Orin NX 16GB pokazuje ~18 tok/s dekodowania dla Llama2 7B Q4_0 na GPU CUDA oraz ~9 tok/s na CPU; zawiera też obserwacje mocy i wpływu flag (np. `-fa 1`). [5]

> **Uwaga o „microGPT"**: to przede wszystkim minimalna implementacja treningu/inferencji małego transformera „od zera" (edukacyjna, a nie gotowy, silny model dialogowy). Nadaje się do demonstracji i ewentualnie do bardzo wąskich modeli znakowych, ale nie jest realnym kandydatem na „mózg robota" w produkcji. [6]

Tabela porównawcza kandydatów

Kandydat	Licencja (wagi/kod)	Rozmiar	Kwantyzacja i formaty	Typowe wymagania pamięci (wagi + narzut)	Szacowana latencja / przepustowość (Jetson Orin NX 16GB)	Wspierane frameworki / runtime	Dojrzałość / społeczność (ocena jakościowa)
Qwen3.5-2B	Apache-2.0 [7]	2B [8]	Int4/Int8/FP16 via GGUF (llama.cpp) + AWQ/GPTQ praktycznie dostępne przez ekosystem; kontrola kontekstu konieczna (domyślnie 262k) [9]	**FP16 ~3.7 GiB**, **int4 ~0.93 GiB** (same wagi; + KV-cache zależny od kontekstu) *(estymacja z liczby parametrów)*	Przy int4 i krótkim kontekście: **~40–80 tok/s**, TTFT typowo **<0.5 s** dla krótkiego promptu *(skalowanie względem 7B Q4_0 na Orin NX)* [10]	Transformers/vLLM/SGLang/KTransformers (oficjalne przykłady serwowania) [11]	**Wysoka** (oficjalne karty modelu + rekomendowane silniki serwujące) [12]
Qwen3.5-4B	Apache-2.0 [13]	4B [13]	Jak wyżej; konieczne ograniczenie kontekstu (262k default). Dodatkowo Qwen3.5 domyślnie generuje „thinking content", co warto wyłączyć dla robota (latencja/ryzyko pętli) [11]	FP16 ~7.45 GiB, int4 ~1.86 GiB + KV-cache *(estymacja)*	**~20–40 tok/s** na GPU dla int4; CPU-only wyraźnie wolniej (kilka–kilkanaście tok/s) [10]	Transformers/vLLM/SGLang/KTransformers; API OpenAI-kompatybilne łatwe do wystawienia [14]	**Wysoka** (duża adopcja ekosystemu Qwen; jasne ścieżki serwowania) [15]
Qwen3.5-9B	Apache-2.0 [16]	9B [17]	Int4 konieczny na 16GB; długi kontekst trzeba zredukować (262k default, deklarowana możliwość rozszerzania) [17]	**FP16 ~16.8 GiB** (praktycznie za dużo), **int4 ~4.2 GiB** + KV-cache *(estymacja)*	**~8–18 tok/s** (int4, krótki kontekst), TTFT zwykle większy niż dla 2–4B *(estymacja na bazie 7B Q4_0)* [10]	Transformers/vLLM/SGLang/KTransformers [18]	**Wysoka**, ale większe ryzyko OOM/thermal throttling na robocie
Microsoft Phi-3-mini-4k-instruct	MIT [19]	3.8B (wprost w karcie jako „3.8B-param") [19]	Oficjalnie opisane: ONNX Runtime (w tym int4 przez AWQ/RTN) + GGUF dla CPU; flash-attention domyślnie [19]	FP16 ~7.1 GiB, int4 ~1.77 GiB + KV-cache *(estymacja)*	Podobnie do 4B: **~20–40 tok/s** (int4) na GPU; plus atut: gotowe artefakty ONNX/int4 [20]	Transformers, ONNX Runtime (cross-platform), GGUF [19]	**Bardzo wysoka** (MIT + szeroki ekosystem, ONNX)
Meta Llama-3.2-3B-Instruct	Licencja społecznościowa Meta + AUP (warunki komercyjne) [21]	3.21B [21]	Dostępne liczne konwersje GGUF/int4 (ekosystem); uwaga na zgodność licencji i dystrybucję [22]	FP16 ~6.0 GiB, int4 ~1.5 GiB *(estymacja)*	**~25–45 tok/s** (int4) na GPU *(estymacja)* [10]	Transformers + „original llama codebase" [21]	**Bardzo wysoka** technicznie, **średnia** prawnie (restrykcje licencji) [21]

Kandydat	Licencja (wagi/kod)	Rozmiar	Kwantyzacja i formaty	Typowe wymagania pamięci (wagi + narzut)	Szacowana latencja / przepustowość (Jetson Orin NX 16GB)	Wspierane frameworki / runtime	Dojrzałość / społeczność (ocena jakościowa)
Mistral-7B-Instruct-v0.2	Apache-2.0 [23]	7B [24]	Bardzo bogaty ekosystem quantów (GGUF/GPTQ/AWQ w społeczności); model jawnie bez mechanizmów moderacji [24]	FP16 ~13.0 GiB, int4 ~3.26 GiB *(estymacja)*	Zbliżone do 7B z benchmarku: **~12–20 tok/s** (int4) na Orin NX GPU [10]	Transformers + liczne serwery (community) [23]	**Wysoka**, ale wymaga warstwy safety (brak moderacji) [25]
Google Gemma 2 (2B/2B-IT)	Gemma Terms of Use (niestandardowe) [26]	~2B	Dla lokalnego uruchamiania: **gemma.cpp** (minimalistyczny C++ runtime) + ekosystem GGUF (llama.cpp) [27]	Zależnie od wariantu/formatu; w GGUF Q4 często **~1–1.5 GiB** dla 2B-klasy *(orientacyjnie, zależne od konkretnego pliku)* [28]	W praktyce szybkie na CPU/GPU dzięki wyspecjalizowanym runtime'om, ale tu decyduje prawo/licencja i łatwość dystrybucji [29]	gemma.cpp, Transformers, GGUF [30]	**Wysoka** technicznie, **średnia** operacyjnie (akceptacja licencji, gating)
NVIDIA PersonaPlex-7B-v1 (speech-to-speech, full-duplex)	Wagi: NVIDIA Open Model License (gated), kod: MIT [31]	7B [32]	Brak „typowej" kwantyzacji w karcie; repo przewiduje **CPU offload** przy braku VRAM (accelerate) [33]	Wagi klasy 7B + dodatkowe komponenty audio; realnie wymaga mocnego GPU lub kompromisów (offload) [34]	Latencje z benchmarku FullDuplexBench są niskie (rzędu setek ms), ale raportowane na mocnym sprzęcie; na Jetson spodziewany duży spadek [32]	PyTorch + serwer moshi [34]	**Wysoka** naukowo (świeże badania), **średnia** wdrożeniowo na edge (ciężar + licencja)
microGPT (Karpathy, „art project")	Brak „standardowej" licencji modelu; to 1-plikowy kod edukacyjny [6]	„mikro" (zależnie od ustawień)	Nie dotyczy typowych quantów LLM; to eksperyment/trening od zera [35]	Minimalne; działa nawet bez numpy/pytorch [6]	Nieporównywalne z LLM dialogowymi; **niezalecane** jako moduł sterowania robota	Pure Python (bez zależności) [35]	**Niska** jako produkt LLM; **wysoka** jako materiał dydaktyczny

Wniosek porównawczy: jeżeli celem jest „realny" asystent na robocie, to najbardziej wdrażalne i przewidywalne są **Qwen3.5-2B/4B (int4)** lub **Phi-3-mini (int4/ONNX)**. Modele 7B+ (Mistral, PersonaPlex, Qwen3.5-9B) są możliwe, ale rosną ryzyka: termika, konkurencja o RAM z percepcją (wideo), niestabilność latencji i OOM.

Kompatybilność sprzętowo-systemowa Unitree G1

Co wiemy z publicznie dostępnych źródeł

Wersje/konfiguracje G1 bywają różne, ale z perspektywy uruchomienia LLM kluczowe są trzy fakty:

Po pierwsze, G1 ma zamkniętą warstwę sterowania lokomocją („locomotion computer", czarna skrzynka) i oddzielny komputer do uruchamiania aplikacji użytkownika („development computer"). To na tym drugim należy uruchamiać LLM i całą logikę aplikacyjną. [1]

Po drugie, w przewodniku developerskim wskazano, że **development computer używa modułu NVIDIA Jetson Orin NX na dedykowanej płycie Unitree**, z własnym BSP (nie flashować obrazami stron trzecich). [1]

Równolegle w polskich materiałach sprzedażowych G1 pojawia się informacja o **8-rdzeniowym CPU** i opcjonalnych wariantach z **NVIDIA Jetson Orin**. [36]

Po trzecie, cechy Jetson Orin NX 16GB istotne dla LLM: 16GB LPDDR5 (128-bit), przepustowość pamięci **~102 GB/s**, CPU do 8 rdzeni, GPU Ampere (1024 CUDA, 32 Tensor), a tryby mocy dla 16GB obejmują **10W/15W/25W** (w praktyce na robotach często liczy się headroom termiczny i stabilność zasilania). [37]

Sieć, dostęp, bezpieczeństwo operacyjne

W G1 urządzenia są spięte przez wewnętrzny przełącznik L2; domyślnie nie ma routera i DHCP, a więc konfiguracja IP jest statyczna. Przykładowo: locomotion computer bywa pod 192.168.123.161, development computer pod 192.168.123.164, a domyślne konto to `unitree/123` (wprost zalecone jest natychmiastowe zmienienie hasła przy podłączaniu do Internetu). [1]

Z punktu widzenia wdrożenia LLM oznacza to, że „zdalny offload" (edge server) jest wykonalny, ale trzeba go świadomie zaprojektować (routing, firewall, segmentacja), bo włączenie Internetu dla podsieci robota może objąć także komponenty krytyczne. [1]

Percepcja i I/O (mowa, wizja) oraz ograniczenia energetyczne

Materiały opisujące G1 (w tym PL) wskazują typowe sensory i multimedia: m.in. czujniki/algorytmy percepcyjne, elementy audio (np. głośnik) i łączność bezprzewodową; w manualach i kartach produktu przewijają się również elementy typu kamera głębi / lidar w zależności od wariantu. [38]

W praktyce dla LLM największym ograniczeniem nie jest „czy CPU istnieje", tylko **czy mamy stabilny budżet mocy i termikę**: Jetson Orin NX ma tryby 10–25W, a przy obciążeniu GPU + wideo + ASR łatwo wejść w throttling, co powoduje jitter latencji (a to jest zabójcze dla interakcji człowiek–robot). [39]

Elementy, których publicznie nie da się potwierdzić

Jeżeli nie masz dokładnej specyfikacji Twojego egzemplarza G1, to **pozostają niepewne**: dokładna wersja JetPack/Ubuntu na komputerze developerskim, typ nośnika (NVMe/eMMC), dostępna przestrzeń dyskowa na modele, oraz czy kamera RGB-D jest podpięta do locomotion computer czy development computer (w przewodniku wspomniano, że zależało to od partii produkcyjnej). [1]

W raporcie dalej przyjmuję **domyślne, konserwatywne założenie**: development computer = Jetson Orin NX 16GB i środowisko Jetson Linux/JetPack (ARM64 + CUDA), ale każdorazowo opisuję, co zrobić, gdy to się nie potwierdzi.

Rekomendacja modeli i konfiguracji docelowej

Wybór modelu bazowego

Rekomendacja bazowa („domyślny mózg rozmowy”): Qwen3.5-4B w int4, z ograniczonym kontekstem i wyłączonym „thinking content”.

Uzasadnienie: Qwen3.5-4B ma Apache-2.0 (proste w dystrybucji komercyjnej), jest wystarczająco mały, by w kwantyzacji 4-bit zmieścić się komfortowo w 16GB klasy Orin NX (waga rzędu kilku GiB + KV-cache), a jednocześnie daje znaczący skok zdolności względem 2B przy nadal sensownej latencji. [40]

Ponieważ Qwen3.5 domyślnie operuje w trybie „thinking” (wstawia blok `<think>…</think>`), dla robota zalecam **tryb non-thinking** jako domyślny (mniej tokenów, mniejsza latencja, mniejsze ryzyko pętli generacji). Sam model ostrzega też, że w mniejszym wariancie 0.8B mogą pojawiać się „thinking loops”, co w systemie robotycznym trzeba traktować jako realne ryzyko zawieszenia dialogu. [11]

Rekomendacja „lekka” (gdy liczy się czas reakcji i zasoby na percepcję): **Qwen3.5-2B w int4.**

Ten wariant jest bardzo atrakcyjny, gdy równolegle uruchamiasz ASR/TTS/wizję, a robot ma zachować płynność odpowiedzi. Przy założeniu podobnej optymalizacji jak w benchmarku 7B Q4_0 na Orin NX, 2B da zwykle kilkadziesiąt tok/s i niską latencję pierwszego tokena. [41]

Rekomendacja alternatywna (silna przenośność + gotowe ONNX/int4): **Phi-3-mini-4k-instruct** (MIT).

Phi-3 ma bardzo mocną stronę operacyjną: w karcie modelu opisano wsparcie ONNX Runtime na wielu platformach oraz gotowe konfiguracje int4 (m.in. AWQ/RTN w kontekście ONNX). To bywa przewagą na embedded, gdy chcesz uniknąć ciężkiego stosu Pythona albo chcesz standaryzować pipeline pod ONNX. [19]

Kiedy w ogóle rozważać większy model

Qwen3.5-9B (int4) ma sens jako tryb „heavy” dla zadań wymagających dłuższego rozumowania albo lepszego planowania narzędzi, ale tylko jeśli:

a) ograniczysz kontekst (KV-cache),

b) zapewnisz stabilny budżet termiczny,

c) rozdzielisz zasoby GPU między percepcję a LLM (QoS),

d) masz twarde timeouts i fallback.

Qwen3.5-9B ma domyślnie 262k kontekstu, co na urządzeniu edge jest praktycznie proszeniem się o OOM, jeśli pozostawisz ustawienia default. [42]

Warstwa bezpieczeństwa

Jeżeli rozważasz Mistral-7B jako model bazowy (bo Apache-2.0 i dojrzałość), pamiętaj, że karta modelu wprost mówi o **braku mechanizmów moderacji**. W robotyce oznacza to konieczność dodatkowej warstwy „policy + gating", niezależnie od tego, jak dobry jest model. [24]

Projekt integracji: architektura, przepływ, API, fuzja sensorów, RT i safety

Założenia architektoniczne dla robota humanoidalnego

Warstwa LLM powinna być projektowana jako **komponent o miękkich wymaganiach czasowych** (soft real-time) i **nigdy** nie może być jedynym źródłem prawdy o bezpieczeństwie ruchu. Zamiast tego LLM „wybiera intencję", a wykonanie odbywa się przez deterministyczne kontrolery/skills.

W Unitree G1 komunikacja z warstwą lokomocji i API odbywa się przez DDS (CycloneDDS), a integracja z ROS2 jest możliwa, bo ROS2 i CycloneDDS są kompatybilne na poziomie DDS. Jest to kluczowe, bo pozwala potraktować LLM jako kolejny węzeł (node) w grafie ROS2, ale bez naruszania wewnętrznej kontroli stabilizacji. [43]

Kanały danych, fuzja sensorów i reprezentacja stanu

W praktyce nie „wpychasz" do LLM surowych danych czujników. Budujesz **warstwę percepcji i streszczeń stanu**, np.:

- **Mowa**: ASR zamienia audio → tekst + confidence + timestamps; dodatkowo VAD (wykrycie mowy) i diarization (opcjonalnie).

- **Wizja**: detekcje obiektów / semantyka sceny / „co robot widzi” w formie krótkich faktów; przy ograniczonym sprzęcie lepiej robić to lekkim modelem i podawać streszczenie do LLM, a nie uruchamiać ciężki VLM.
- **Propriocepcja**: pozycje stawów, prędkości, stan kontaktu stóp, tryb (normal/debug), ograniczenia, alarmy (przegrzanie, niskie napięcie).
- **„World model” (pamięć robota):** aktualny cel, kontekst zadania, mapka topologiczna, lista „dozwolonych czynności”.

Interfejsy API pomiędzy LLM, a sterowaniem robota

Zalecany wzorzec to **tool-calling / function-calling**, ale z rygorem: LLM nie wysyła „surowych komend ruchu”, tylko wywołuje zdefiniowane narzędzia o małym zakresie i weryfikowalnych parametrach.

Przykładowy kontrakt narzędzi (idee):

- `say(text, voice_id, priority)`
- `ask_clarification(question, options)`
- `navigate_to(pose, speed_profile)`
- `manipulate(object_id, grasp_type, constraints)`
- `set_posture(mode)`
- `stop_all(reason)` (zawsze dozwolone)

Każde narzędzie przechodzi przez **Safety Gate**: walidacja schematu, limity prędkości/sił, strefy zakazane, kontrola stanu (czy robot jest stabilny), oraz **timeout** (jeśli nie dostaniemy potwierdzenia z kontrolera w X ms → awaryjne stop).

Tryby awaryjne i bezpieczeństwo funkcjonalne

W dokumentacji G1 występuje pojęcie przełączenia w tryb debug (pełna kontrola użytkownika). To jest przydatne do badań, ale dla systemu opartego o LLM trzeba to traktować jako „tryb wysokiego ryzyka” i ograniczyć dostęp (np. tylko operator). [1]

Dodatkowo, G1 ma domyślne ustawienia sieci i hasła, które są słabe; jeżeli robot ma mieć Internet, konieczna jest zmiana hasła i twarde reguły firewall. [1]

Diagram architektury

```
flowchart TB
  subgraph G1["Unitree G1 (system)"]
    LC["Locomotion Computer (zamknięty)\n- stabilizacja\n- sterowanie motoram
```

```
i\n- sensory low-level"]:::closed
    DC["Development Computer (dostępny)\nJetson Orin NX (typowo)\nLinux/ARM64
+ CUDA"]:::open
  end

  subgraph Perception["Percepcja i stan"]
    ASR["ASR/VAD\n(audio->tekst + confidence)"]:::svc
    VISION["Wizja (lekka)\n(det/seg/scene->fakty)"]:::svc
    PROPRIO["Propriocepcja\n(IMU/stawy/tryb/alerty)"]:::svc
    WM["World Model\n(pamięć robota, cele, kontekst)"]:::svc
    SUM["State Summarizer\n(twarde ograniczenia + streszczenie)"]:::svc
  end

  subgraph LLMStack["Warstwa LLM"]
    LLM["LLM lokalny\n(Qwen3.5-2B/4B int4)\nllama.cpp/MLC/ONNX"]:::llm
    POLICY["Safety & Policy Gate\n- schematy\n- limity\n- timeouts\n- reguły"
]:::safety
    TOOL["Tool Router\n(mapowanie -> skills)"]:::svc
  end

  subgraph Control["Kontrola (poza LLM)"]
    SKILLS["Skill Executor\n(nawigacja, gesty, chwyt)\nROS2/DDS klient"]:::ct
rl
    MON["Monitor i watchdog\n(health, FPS, temp, RAM)\nrollback/fallback"]:::
svc
  end

  ASR --> SUM
  VISION --> SUM
  PROPRIO --> SUM
  WM --> SUM
  SUM --> LLM
  LLM --> POLICY
  POLICY --> TOOL
  TOOL --> SKILLS

  SKILLS <--> DC
  DC <--> LC

  MON --> POLICY
  MON --> LLM
  MON --> SKILLS

  classDef closed fill:#eee,stroke:#999,color:#333;
  classDef open fill:#e8f4ff,stroke:#2b6cb0,color:#123;
  classDef llm fill:#fff4e6,stroke:#c05621,color:#111;
  classDef safety fill:#ffe6e6,stroke:#c53030,color:#111;
  classDef ctrl fill:#e6ffed,stroke:#2f855a,color:#111;
  classDef svc fill:#f7fafc,stroke:#4a5568,color:#111;
```

Optymalizacja i plan wdrożenia

Optymalizacje modelu i inferencji

Największe „dźwignie” na embedded to:

Kwantyzacja wag: dla llama.cpp standardem jest GGUF. Narzędzie `quantize` bierze model GGUF w wysokiej precyzji (F32/BF16) i konwertuje do formatów niskobitowych, redukując rozmiar i często przyspieszając inferencję; jakość można poprawiać użyciem `imatrix` (kalibracja). [44]

Dla modeli Qwen istnieje też oficjalna ścieżka kwantyzacji GGUF obejmująca m.in. możliwość zastosowania skali AWQ. [45]

Ograniczenie kontekstu: Qwen3.5 ma domyślny kontekst 262k; na robocie należy ustawić realny limit (np. 8k–16k) i świadomie zarządzać pamięcią rozmowy (streszczanie, pamięć epizodyczna). W przeciwnym razie KV-cache wciągnie RAM i doprowadzi do OOM. [46]

Silniki inferencji:

- **llama.cpp**: świetne dla GGUF + prosty serwer/CLI; ma praktyczne mechanizmy kwantyzacji i bogaty ekosystem. [44]
- **MLC LLM**: kompiluje/uruchamia modele na wielu backendach (CUDA/Vulkan/Metal; dokumentacja opisuje też konfigurację GPU i SDK). To bywa atrakcyjne na embedded, bo potrafi dać dobre osiągi bez ręcznego strojenia kernelami. [47]
- **ONNX Runtime**: szczególnie sensowne dla Phi-3, bo karta modelu opisuje gotowe warianty ONNX/int4 i wsparcie cross-platform. [19]

Strojenie na Jetson: Orin NX ma tryby mocy 10/15/25W i (zależnie od JetPack) tryby „SUPER”; w praktyce musisz dobrać profil tak, by uniknąć throttlingu podczas dłuższych dialogów + percepcji. Sama społeczność llama.cpp raportuje, że na Orin NX 16GB w trybie wysokiej wydajności uzyskuje ~18 tok/s dla 7B Q4_0 na GPU, ale też wskazuje na wrażliwość na ustawienia i ograniczenia architektury (np. brak FP8 na Ampere). [48]

Offload i degradacja jakości: PersonaPlex wprost dokumentuje `--cpu-offload` (dla braku VRAM), co jest analogiczną strategią do „layer offload” w innych runtime’ach — użyteczne jako bezpiecznik, ale kosztuje latencję. [33]

Plan deploymentu: pakowanie, kontenery, CI/CD, testy, monitoring, rollback

Ponieważ development computer jest elementem systemu robota i ma ograniczenia BSP, zalecam podejście „minimalnie inwazyjne”:

Pakowanie: - Preferuj kontenery ARM64 z NVIDIA Container Runtime (na Jetson), ale nie zakładaj, że zawsze da się użyć gotowych obrazów (różnice JetPack). JetPack 6 to Ubuntu 22.04 + kernel 5.15, ale G1 bywa spotykany też na JetPack 5.1.x/Ubuntu 20.04 — to wpływa na binaria i zależności. [49] - Modele duże trzymaj na szybkim nośniku (NVMe). W datasheet Orin NX wspomina się o wsparciu dla zewnętrznego NVMe przez PCIe, co jest praktycznie jedyną sensowną drogą na duże GGUF. [50]

Cross-kompilacja i build: - Dla runtime’ów wymagających kompilacji (np. llama.cpp, MLC) buduj artefakty w CI z `docker buildx` na ARM64. - Jeżeli rozważasz vLLM: jest wsparcie dla ARM64 CPU (NEON), ale zwykle brak gotowych wheel’i — trzeba budować ze źródeł i liczyć się z większym narzutem. [51]

Testy:

- Testy jednostkowe: walidacja schematów narzędzi, timeouts, parser odpowiedzi LLM.
- Testy integracyjne: replay sesji (nagrania audio+telemetria) → oczekiwane narzędzia i brak komend zakazanych.
- Testy „hardware-in-the-loop”: uruchamiaj na robocie tylko w trybie bezpiecznym (ograniczone prędkości, miękkie stany), z fizycznym E-stopem/operator mode.

Monitoring:

- Metryki LLM: tok/s, TTFT (time-to-first-token), zużycie RAM/GPU, OOM count, długości odpowiedzi.
- Metryki robota: temperatury, napięcie, watchdog DDS/ROS2, liczba „stop_all”.
- Na Jetson można skorzystać z gotowych usług monitoringu/integracji API gateway (NVIDIA Jetson Platform Services pokazuje wzorzec: kontener, REST API, integracja z monitoringiem i ingress). [52]

Rollback:

- Wersjonuj modele i runtime osobno (np. `llm-runtime:vX`, `model-pack:qwen35-4b-q4km-vY`).

- W przypadku wzrostu błędów/jitteru: automatyczny powrót do mniejszego modelu (2B) albo do trybu „bez LLM” (reguły + predefiniowane komendy).

Gantt wdrożenia (propozycja)

```
gantt
  title Rollout LLM lokalnie na Unitree G1 (propozycja)
  dateFormat  YYYY-MM-DD
  axisFormat  %d.%m

  section Przygotowanie
  Audyt sprzętu/JetPack i ograniczeń BSP     :a1, 2026-03-06, 5d
  Dobór modeli + formatów (GGUF/ONNX)        :a2, after a1, 5d

  section MVP (tekstowy asystent)
  Integracja DDS/ROS2 + skill API            :b1, 2026-03-16, 10d
  Uruchomienie LLM (Qwen3.5-2B/4B int4)      :b2, after b1, 7d
  Safety Gate + timeouts + fallback          :b3, after b2, 7d

  section Percepcja i UX
  ASR/TTS pipeline + synchronizacja          :c1, 2026-04-09, 10d
  State summarizer + pamięć rozmowy          :c2, after c1, 7d

  section Optymalizacja i stabilizacja
  Profilowanie (tok/s, TTFT, termika)        :d1, 2026-04-28, 7d
  Optymalizacje (kontekst, kwantyzacja, QoS) :d2, after d1, 10d

  section Wdrożenie produkcyjne
  Monitoring/alerting + procedury rollback    :e1, 2026-05-15, 7d
  Testy HIL + scenariusze bezpieczeństwa      :e2, after e1, 10d
  Pilotaż na robocie (operator-in-loop)       :e3, after e2, 10d
```

Bezpieczeństwo, prywatność, licencje oraz estymacja wysiłku i ryzyka

Bezpieczeństwo i prywatność (robot w świecie fizycznym)

Najważniejsze ryzyko LLM w robotyce to nie „halucynacje tekstowe”, tylko **halucynacje akcji**. Z tego powodu:

- LLM nie może mieć bezpośredniego kanału do momentów/sił; tylko do prymitywów z limitami.
- Wymuszaj „bezpieczny język komend” i schematy. Odpowiedź niezgodna ze schematem = odrzucenie + prośba o doprecyzowanie.

- Wprowadź watchdog: jeśli LLM nie odpowiada w czasie, system przechodzi w stan „bezpieczny” i dalej działa deterministycznie.
- Ogranicz sieć: ponieważ robot ma statyczne IP i brak DHCP/routera, każde „wystawienie” API LLM wymaga świadomej konfiguracji podsieci; dokumentacja ostrzega o słabym haśle domyślnym i zaleca jego zmianę przy dostępie do Internetu. [1]

Licencje i zgodność

- **Qwen3.5**: Apache-2.0 (wagi i konfiguracje w HF). To zwykle najprostsza ścieżka do komercyjnego wdrożenia lokalnego. [53]
- **Phi-3-mini**: MIT. Bardzo „czysta” licencja, dobra do produktów. [19]
- **Mistral-7B-Instruct-v0.2**: Apache-2.0, ale brak moderacji wymusza własne guardrails (to bardziej kwestia bezpieczeństwa niż prawa). [54]
- **Llama-3.2**: licencja społecznościowa Meta + Acceptable Use Policy; technicznie mocna opcja, ale prawnie bardziej złożona (szczególnie przy redystrybucji artefaktów). [21]
- **Gemma**: niestandardowe Terms of Use Google; często wymaga akceptacji warunków (gating) — to zwiększa koszty operacyjne (zgodność, procesy, audyt). [26]
- **PersonaPlex**: wagi są gated i objęte NVIDIA Open Model License; karta HF wskazuje konieczność akceptacji warunków i udostępnienia danych kontaktowych. To może być problematyczne w projektach, które wymagają w pełni „niezależnej” dystrybucji. [31]
- **microGPT**: to kod edukacyjny (art project), a nie standardowy „model do wdrożenia”; z punktu widzenia compliance i ryzyk produktu zwykle odpada. [6]

Estymacja wysiłku implementacyjnego

Zakładam zespół 2 osoby (robotyka/ROS2 + ML/infra) i korzystanie z gotowych runtime’ów (llama.cpp lub ONNX).

Bardzo zgrubnie:

- MVP tekstowy (LLM + tool-calling + kilka skills + safety gate): **4–6 tygodni**.
- Dodanie ASR/TTS i spójnej synchronizacji dialogu: **+2–4 tygodnie**.
- Wersja „produkcyjna” z monitoringiem, testami HIL, rollback i twardymi procedurami bezpieczeństwa: **+4–8 tygodni**.
- PersonaPlex jako full-duplex voice agent (jeśli w ogóle na G1): **+6–10 tygodni** i wysokie ryzyko, bo model jest celowany w większe GPU, a repo przewiduje co prawda offload, ale kosztem latencji. [34]

Macierz ryzyka (inżynierska)

Ryzyko	Prawdopodobieństwo	Wpływ	Objawy	Mitigacje
OOM / niestabilna pamięć (KV-cache, długi kontekst)	Wysokie (domyślne 262k w Qwen3.5) [55]	Wysoki	zabicia procesu, restart, freeze dialogu	limit kontekstu 8k–16k; streszczanie; stałe limity tokenów; watchdog
Throttling termiczny → jitter latencji	Średnie–wysokie [39]	Wysoki	skoki TTFT/tok/s, zrywanie dialogu	profil mocy (nvpmodel), limity FPS wizji, priorytety GPU, fallback do 2B
Niebezpieczne akcje z LLM (hallucinated tool calls)	Średnie	Krytyczny	ruch w złym kierunku, nieprzewidywalne gesty	skill-gating, walidacja schematów, limity prędkości/sił, operator mode, „stop_all"
Ryzyko sieciowe (domyślne hasło, ekspozycja podsieci robota)	Średnie [1]	Wysoki	nieautoryzowany dostęp, sterowanie	zmiana haseł, firewall, brak publicznego API, segmentacja, VPN
Złożoność BSP na G1 (nie flashować, zależności JetPack)	Średnie [1]	Wysoki	brick systemu, brak sterowników	minimalna inwazyjność, kontenery, snapshoty, polityka aktualizacji
Compliance/licencje (gating, restrykcje dystrybucji)	Średnie	Średni–wysoki	blokada prawna release'u	preferuj Apache-2.0/MIT; audyt licencji i artefaktów; unikać gated wag w core
Brak moderacji w modelu (np. Mistral)	Wysokie [25]	Średni–wysoki	toksyczne treści, błędne instrukcje	dodatkowy filtr treści + polityka dialogu; separacja „mowy" od „akcji"

Najważniejsza praktyczna rekomendacja: nawet przy najlepszym modelu (Qwen3.5-4B / Phi-3) kluczowe jest, by **LLM nie sterował ruchem bezpośrednio** oraz by zawsze istniał deterministyczny fallback i twarde ograniczenia bezpieczeństwa, zgodne z rozdziałem „komputer lokomocji jako czarna skrzynka" i DDS/ROS2 jako warstwa integracji. [43]